FSD Assignments — General Instructions

Oct 23, 2026 · @DK

Overview

Every FSD assignment (FSD-01 to FSD-50) asks you to build and deploy a working, role-based full-stack application in **3 hours**. Each one has JWT login, a real database, and one critical business rule that the backend must enforce.

The assignments differ only in the domain, the roles and the business rule. Everything in this document applies to all 50; your assignment sheet adds the specifics.

Each assignment sheet follows the same order:

1. **Objective** — what the system does.
2. **User Roles** — two or three roles and what each may do.
3. **Functional Requirements** — authentication plus the domain features.
4. **Business Rule (Critical)** — the one rule your system must never break.
5. **Technical Constraints (Mandatory)** — persistence, JWT and environment rules.
6. **Frontend Requirements** — the pages and dashboards needed.
7. **Submission Requirements** — repository, live links and README.
8. **Evaluation Note** — what the assessor focuses on (FSD-03 onward).

Tech stack and project setup

No assignment prescribes a framework: you choose the stack and name it in your README. Whatever you pick must cover these fixed parts:

Part	What is required
Frontend	A separate client app with login, registration and role-based dashboards
Backend	A REST API that holds all business logic and validation
Database	A real, persistent database (SQL tables or NoSQL collections)
Authentication	JWT issued at login and checked on every protected API

Part	What is required
Configuration	Secrets (DB URL, JWT secret, keys) in environment variables, never in code
Hosting	Frontend and backend each deployed to a live URL, backend connected to the hosted database

Keep frontend and backend code in a GitHub repository with a clear folder structure. Commit as you go so the history shows how the work progressed.

Common functional requirements

Every assignment requires the same authentication core, then two or three roles working on shared data stored in the database.

Authentication and authorization (all 50 assignments)

- Implement user registration and login.
- Use JWT-based authentication.
- Protect all backend APIs with JWT.
- Enforce role-based access control on the backend APIs **and** on the frontend views.

Roles

- Use the role names exactly as your sheet gives them (for example USER/ADMIN, REQUESTER/AGENT/ADMIN, CREATOR/EXECUTOR).
- Each role may do only what its list allows. A request from the wrong role must be rejected by the API, not just hidden in the UI.

Domain features

- Create, list and update the main records (resources, tickets, bookings, requests and so on) as the sheet describes.
- Store every record and every status change in the database.
- Fetch lists, availability and status dynamically from the database.

The critical business rule

- Enforce it in the backend, against the database, before saving anything.
- Reject a violating request with a clear error message, and show that message in the UI.
- Where the rule depends on time (deadlines, windows, SLAs, edit limits), use server time, not client time.

Backend, database and API rules (mandatory)

These technical constraints appear in every assignment. Breaking any one of them means the assignment does not meet its requirements.

1. **All data must persist in a database.** Users, records, statuses, rules and logs are all stored and read back from the database.
2. **No hardcoded or in-memory storage.** Arrays, JSON files or server variables used as a data store are not allowed, and neither are hardcoded calculations.
3. **APIs must reflect the actual database state.** A refresh or a server restart must show the same data.
4. **JWT authentication and role validation are mandatory** on every protected endpoint.
5. **Sensitive configuration goes in environment variables.** Do not commit secrets; commit a `.env.example` instead.

Recommended practice:

- Validate input and business rules on the server; frontend checks are extra, never the only check.
- Return proper HTTP status codes (400 invalid input, 401 not logged in, 403 wrong role, 404 not found, 409 conflict or rule violation).
- Use database constraints (unique keys, foreign keys) where they back up a rule, such as no duplicate reviews or one vote per poll.
- Keep routes, controllers and models in separate folders so the structure is easy to follow.

Worked example: enforcing a workflow on the backend

This example shows the pattern most assignments test: the backend decides which state changes are allowed, by whom, and rejects everything else. It uses a generic approval request; apply the same idea to your own records, statuses and roles.

States: DRAFT (created, not submitted) · PENDING (awaiting review) · APPROVED · REJECTED · ESCALATED (overdue, moved up) · CLOSED (final, read-only).

Current state	Action	Allowed role	Next state	Rule
DRAFT	Edit request	USER	DRAFT	Edits allowed only before submission
DRAFT	Submit request	USER	PENDING	No edits after submission

Current state	Action	Allowed role	Next state	Rule
PENDING	Approve	REVIEWER	APPROVED	All business rules must pass
PENDING	Reject	REVIEWER	REJECTED	Rejection reason is mandatory
PENDING	Time limit exceeded	SYSTEM	ESCALATED	Triggered automatically using server time
ESCALATED	Approve or reject	ADMIN	APPROVED / REJECTED	Override after escalation
APPROVED / REJECTED	Close	SYSTEM or ADMIN	CLOSED	Record becomes read-only
CLOSED	Any action	Any role	Not allowed	Final state

Changes the API must always reject: DRAFT → APPROVED, DRAFT → REJECTED, PENDING → CLOSED, APPROVED → PENDING, REJECTED → PENDING, and CLOSED → any state.

How to build it:

1. Store the current status on the record in the database.
2. On every action, load the record, then check the current status, the user's role and the business rule on the server.
3. If any check fails, return an error (403 for the wrong role, 409 for a disallowed change) and save nothing.
4. If all checks pass, save the new status and its timestamp, ideally with a history entry of who did what and when.

Frontend requirements

UI design is not evaluated: what matters is that every role can complete its workflow and see the result clearly.

Every frontend must include:

- Login and registration pages.
- A separate dashboard for each role named in your sheet.
- Lists of the main records with their current status, read from the API.

- The actions each role is allowed to take (approve, reject, assign, configure and so on), and only those.
- Clear feedback when the backend rejects an action, especially a business-rule violation.

Also build the extra views your sheet names, such as reports, history, audit logs or configuration screens for ADMIN.

Submission requirements

Submit three things: a GitHub repository, two live links and a complete README. All 50 assignment sheets now use this same strict checklist (section 7, Submission Requirements (Strict)).

GitHub repository

- [ ] Backend source code
- [ ] Frontend source code
- [ ] Proper folder structure
- [ ] Clear, meaningful commit history
- [ ] No secrets committed (`.env` ignored, `.env.example` included)

Live deployment

- [ ] Live frontend link
- [ ] Live backend link
- [ ] Backend connected to a hosted database

README.md (mandatory)

- [ ] Project overview
- [ ] Tech stack used
- [ ] User roles and permissions
- [ ] API endpoints (method, path, role allowed)
- [ ] Database schema (tables or collections)
- [ ] Steps to run the project locally
- [ ] Live frontend and backend links

Test both live links in a private browser window before you submit.

Do's and don'ts

Above all, keep rules on the server and data in the database: a rule checked only in the UI, or data held in memory, misses the core requirement.

Do

- Read the Business Rule first and design your schema and APIs around it.
- Test the rule by calling the API directly (Postman or curl), not only through the UI.
- Test with one account per role.
- Deploy early and redeploy as you go, so the live links work by the end of the 3 hours.
- Commit in small, meaningful steps.

Don't

- Don't store data in arrays, files or local storage instead of the database.
- Don't enforce a rule or a role check only in the frontend.
- Don't trust client-side time or client-sent role values.
- Don't hardcode secrets, users or lists.
- Don't spend time on styling or extra features before the required workflow works end to end.

Assignment index

All 50 assignments, with their roles and the critical business rule each one must enforce.

ID	Title	Roles	Critical business rule
FSD-01	Resource Allocation Management System	USER, ADMIN	A resource can be allocated to only one user at a time
FSD-02	Slot Booking Management System	USER, MANAGER	No overlapping time slots for the same resource
FSD-03	Internal Issue Tracking System	EMPLOYEE, ADMIN	Status must go OPEN → IN_PROGRESS → RESOLVED, no skipping or reverting
FSD-04	Review Moderation Platform	USER, MODERATOR	No duplicate reviews by a user for the same item

ID	Title	Roles	Critical business rule
FSD-05	Access Request Management Portal	REQUESTER, APPROVER	Only one pending access request per user at a time
FSD-06	Inventory Request Management System	STAFF, ADMIN	Requested quantity must not exceed available stock
FSD-07	Event Registration Management System	USER, ORGANIZER	Registration closes automatically at maximum capacity
FSD-08	Internal Announcement Management System	EMPLOYEE, ADMIN	Expired announcements must not be visible to employees
FSD-09	Shift Assignment Management System	STAFF, MANAGER	A staff member must not get overlapping shifts
FSD-10	Candidate Shortlisting Management System	RECRUITER, HR	Status must follow APPLIED → SHORTLISTED → REJECTED; final status cannot change
FSD-11	Submission Review Management System	CONTRIBUTOR, REVIEWER	Maximum two resubmissions per entry
FSD-12	Room Booking Management System	USER, ADMIN	No overlapping bookings for the same room
FSD-13	Internal Polling Management System	USER, ADMIN	One vote per user per poll
FSD-14	Feature Request Management System	USER, ADMIN	No duplicate feature request titles
FSD-15	Complaint Management System	USER, SUPPORT	Feedback only after the complaint is CLOSED
FSD-16	Document Access Control System	USER, ADMIN	Download only documents with approved access

ID	Title	Roles	Critical business rule
FSD-17	Approval Workflow Management System	CREATOR, APPROVER	A creator cannot approve their own request
FSD-18	Task Assignment Management System	MANAGER, MEMBER	Deadline cannot be earlier than the assignment date
FSD-19	Meeting Scheduling Management System	ORGANIZER, PARTICIPANT	No overlapping meetings for a participant
FSD-20	Feedback Collection Management System	USER, ADMIN	Feedback editable only within 15 minutes of submission
FSD-21	Internal Knowledge Base Management System	CONTRIBUTOR, ADMIN	Articles must be approved before they are published or visible
FSD-22	Leave Management System	EMPLOYEE, MANAGER	Reject leave when the balance is insufficient
FSD-23	Course Enrollment Management System	STUDENT, INSTRUCTOR	No enrollment after the course start date
FSD-24	Support Ticket Management System	CUSTOMER, AGENT	A ticket can be reassigned only once
FSD-25	Data Access Control Management System	USER, ADMIN	Users access only datasets assigned to them
FSD-26	Internal Asset Tracking Management System	USER, ADMIN	An asset must be returned before reassignment
FSD-27	Interview Slot Management System	CANDIDATE, RECRUITER	One active interview slot per candidate

ID	Title	Roles	Critical business rule
FSD-28	Content Publishing Workflow System	AUTHOR, EDITOR	Published content cannot be edited
FSD-29	Subscription Control Management System	USER, ADMIN	One active subscription per user
FSD-30	Incident Reporting Management System	REPORTER, RESOLVER	Incident priority can never be downgraded
FSD-31	Certification Approval Management System	APPLICANT, VERIFIER	Each request can be verified or rejected only once
FSD-32	Internal Training Enrollment Management System	EMPLOYEE, TRAINER	No enrollment beyond the seat limit
FSD-33	Project Assignment Tracking System	MANAGER, DEVELOPER	A developer cannot be on overlapping projects
FSD-34	Access Revocation Management System	USER, ADMIN	Revoked access can be restored only by ADMIN
FSD-35	Internal Evaluation Management System	EVALUATOR, ADMIN	Submitted scores are final and cannot be modified
FSD-36	Time-Based Access Window System	USER, ADMIN	Access only within the user's assigned time window
FSD-37	Controlled Submission Deadline System	USER, ADMIN	No submissions after the deadline
FSD-38	Quota-Limited Request Management System	USER, ADMIN	Reject requests once the quota is exhausted

ID	Title	Roles	Critical business rule
FSD-39	Delegation-Based Approval Management System	REQUESTER, MANAGER, DELEGATE	A delegate approves only while the delegation is active
FSD-40	Conditional Escalation Management System	REPORTER, HANDLER, ESCALATION_MANAGER	Only ESCALATION_MANAGER can resolve an escalated issue
FSD-41	Multi-Level Review and Sign-Off System	SUBMITTER, REVIEWER_LEVEL_1, REVIEWER_LEVEL_2	Level 2 review only after Level 1 approval
FSD-42	Dependency-Based Task Execution System	CREATOR, EXECUTOR	A task runs only after all its dependencies are completed
FSD-43	Version-Controlled Record Management System	EDITOR, VIEWER	Only the latest version can be edited
FSD-44	Threshold-Based Alert Management System	OPERATOR, ADMIN	Alerts are generated automatically when a threshold is crossed; no manual alerts
FSD-45	Immutable Audit Log Management System	USER, ADMIN	Audit log entries can never be edited or deleted
FSD-46	Rate-Limited API Access Management System	USER, ADMIN	Block protected API calls once the rate limit is exceeded until the window resets
FSD-47	Conflict-Resolved Data Update System	USER, ADMIN	Reject updates to a record changed since it was last read; no silent overwrites
FSD-48	SLA-Based Ticket Aging and Breach Management System	REQUESTER, AGENT, ADMIN	A detected SLA breach is permanent, even after resolution

ID	Title	Roles	Critical business rule
FSD-49	Rule-Based Discount Eligibility Management System	CUSTOMER, ADMIN	Apply a discount only if all eligibility rules are met
FSD-50	Data Retention and Archival Policy Management System	USER, ADMIN	Archived records cannot be edited or restored